

Week-20 | Azure Key Vault Integration with AKS | Kubernetes Secret Store CSI Driver with Azure Vault



Introduction

Managing secrets securely is a crucial aspect of application development, especially in cloud environments. Kubernetes provides a built-in secret management system, but it has limitations in terms of security, compliance, and lifecycle management. Azure Key Vault (AKV) serves as a centralized, secure, and scalable solution for managing secrets, keys, and certificates.

By integrating **Azure Key Vault with Azure Kubernetes Service (AKS)** using the **Secret Store Container Storage Interface (CSI) driver**, organizations can securely inject secrets into their Kubernetes workloads without hardcoding sensitive information into application configurations or CI/CD pipelines.

This document provides a comprehensive guide to setting up **Azure Key Vault integration with AKS** using the **Secret Store CSI Driver**. It includes theoretical insights, practical benefits, and a detailed step-by-step implementation using Azure CLI and Kubernetes YAML configurations.

Benefits of Using Azure Key Vault for Secret Management



1. Centralized Secrets Management

- Stores secrets, encryption keys, and certificates in one place.
- Simplifies security management across multiple applications and environments.
- Enables logging and monitoring of secret access via **Azure Monitor and Azure Security Center**.

2. Security and Compliance

- **RBAC (Role-Based Access Control)** ensures granular access to secrets.
- **Automatic secret rotation** prevents credential leaks and ensures compliance with security policies.
- **Integration with Azure Active Directory (Azure AD)** enables managed identities for authentication.

3. Secure Kubernetes Secrets Management

- Kubernetes Secrets are stored in **etcd**, which may not be encrypted by default.
- Azure Key Vault provides **built-in encryption** using **HSM-backed keys** and compliance with security standards.
- **No need to store secrets in Kubernetes manifests or ConfigMaps.**

4. Workload Identity Integration

- Eliminates the need to store credentials inside pods or use Kubernetes service accounts.
- Uses **Azure Managed Identities** for authentication without embedding credentials in container images.

5. DevOps and CI/CD Integration

- Enhances **security in CI/CD pipelines** by retrieving secrets dynamically instead of hardcoding them.
- Works seamlessly with **GitHub Actions, Azure DevOps, Terraform, and Ansible.**

Setting Up AKS with Azure Key Vault and CSI Driver

This guide will cover:

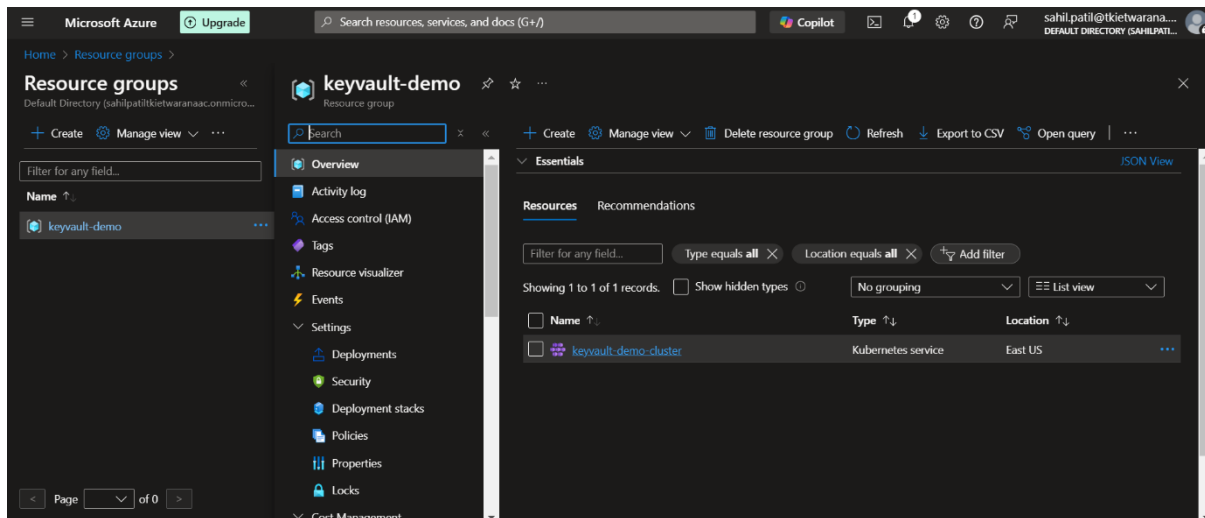
1. Creating an **Azure Resource Group** and **AKS Cluster**.
2. Configuring **Azure Key Vault**.
3. **Setting up Workload Identity** for secure authentication.
4. Deploying **Kubernetes secrets integration with AKV using the CSI driver**.
5. **Verifying the integration** with a test pod.
6. Cleanup of resources.

Step 1: Create an Azure Resource Group

```
az group create --name keyvault-demo --location eastus
```

```
AzureAD+SahilShivajiPatil@DESKTOP-DBH45CS MINGW64 -
$ az group create --name keyvault-demo --location eastus
{
  "id": "/subscriptions/8809e93f-1006-482f-9f2d-f3b60a303da5/resourceGroups/keyvault-demo",
  "location": "eastus",
  "managedBy": null,
  "name": "keyvault-demo",
  "properties": {
    "provisioningState": "Succeeded"
  },
  "tags": null,
  "type": "Microsoft.Resources/resourceGroups"
}

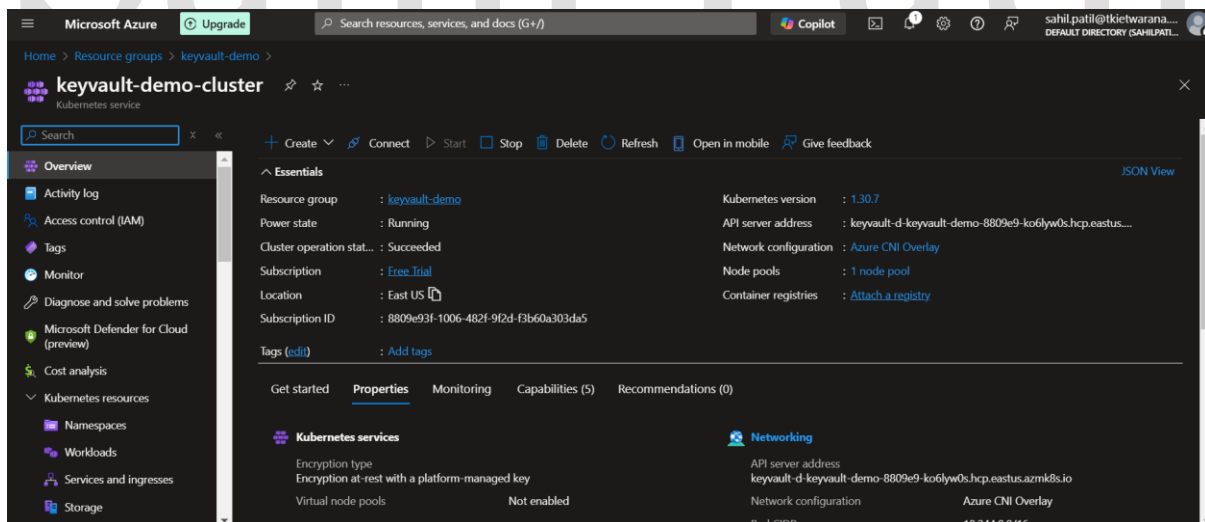
AzureAD+SahilShivajiPatil@DESKTOP-DBH45CS MINGW64 -
$ az aks create --name keyvault-demo-cluster -g keyvault-demo --node-count 1 --enable-addons azure-keyvault-secrets-provider --enable-oidc-issuer --enable-workload-identity
{
  "aadProfile": null,
  "addonProfiles": {
    "azureKeyvaultSecretsProvider": {
      "config": {
        "enableSecretRotation": "false",
        "rotationPollInterval": "2m"
      },
      "enabled": true,
      "identity": {
        "clientId": "12eb6f64-ec8f-4509-bd7e-f97e4a116052",
        "objectId": "8236bce3-41a2-45ad-9aab-744907faff54",
        "resourceId": "/subscriptions/8809e93f-1006-482f-9f2d-f3b60a303da5/resourceGroups/MC_keyvault-demo_keyvault-demo-cluster_eastus/providers/Microsoft.ManagedIdentity/userAssignedIdentities/azurekeyvaultsecretsprovider-keyvault-demo-cluster"
      }
    }
  },
  "agentPoolProfiles": [
    {
      "availabilityZones": null,
      "capacityReservationGroup": null,
      "count": 1,
      "creationData": null,
      "currentOrchestratorVersion": "1.30.7",
      "etag": null,
      "enableAutoscaling": false,
      "enableEncryptionAtHost": false,
      "enableFips": false,
      "enableNodePublicIp": false,
    }
  ]
}
```



This creates a **resource group** where all Azure resources will be managed.

Step 2: Create an AKS Cluster with Azure Key Vault Support

```
az aks create --name keyvault-demo-cluster -g keyvault-demo --node-count 1 \
--enable-addons azure-keyvault-secrets-provider \
--enable-oidc-issuer --enable-workload-identity
```



Explanation:

- `--enable-addons azure-keyvault-secrets-provider`: Enables the **Secrets Store CSI Driver**.
- `--enable-oidc-issuer`: Enables OpenID Connect (OIDC) for authentication.
- `--enable-workload-identity`: Enables **Workload Identity**, replacing Kubernetes service accounts with Azure identities.

Step 3: Connect to the AKS Cluster

```
az aks get-credentials --resource-group keyvault-demo --name keyvault-demo-cluster
```

Verify the connection:

```
kubectl get nodes
```

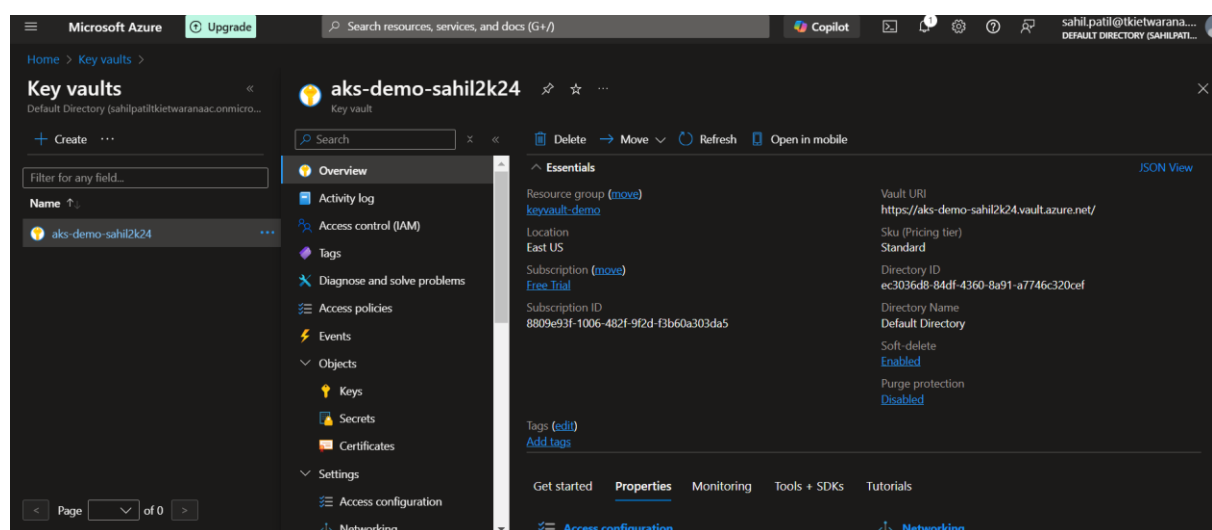
Check if the **Secrets Store CSI Driver** is running:

```
kubectl get pods -n kube-system -l 'app in (secrets-store-csi-driver,secrets-store-provider-azure)' -o wide
```

Step 4: Create Azure Key Vault

```
az keyvault create -n aks-demo-abhi -g keyvault-demo -l eastus --enable-rbac-authorization
```

```
AzureAD+SahilShivajiPatil@DESKTOP-DBH45CS MINGW64 ~
$ az keyvault create -n aks-demo-sahil2k24 -g keyvault-demo -l eastus --enable-rbac-authorization
{
  "id": "/subscriptions/8809e93f-1006-482f-9f2d-f3b60a303da5/resourceGroups/keyvault-demo/providers/Microsoft.KeyVault/vaults/aks-demo-sahil2k24",
  "location": "eastus",
  "name": "aks-demo-sahil2k24",
  "properties": {
    "accessPolicies": [],
    "createMode": null,
    "enablePurgeProtection": null,
    "enableRbacAuthorization": true,
    "enableSoftDelete": true,
    "enabledForDeployment": false,
    "enabledForDiskEncryption": null,
    "enabledForTemplateDeployment": null,
    "hsmPoolResourceId": null,
    "networkAcls": null,
    "privateEndpointConnections": null,
    "provisioningState": "Succeeded",
    "publicNetworkAccess": "Enabled",
    "sku": {
      "family": "A",
      "name": "standard"
    },
    "softDeleteRetentionInDays": 90,
    "tenantId": "ec3036d8-84df-4360-8a91-a7746c320cef",
    "vaultUri": "https://aks-demo-sahil2k24.vault.azure.net/"
  },
  "resourceGroup": "keyvault-demo",
  "systemData": {
    "createdAt": "2025-01-29T16:18:32.340000+00:00",
    "createdBy": "sahil.patil@tkietwarana.ac.in",
    "createdByType": "User",
    "lastModifiedAt": "2025-01-29T16:18:32.340000+00:00",
    "lastModifiedBy": "sahil.patil@tkietwarana.ac.in",
    "lastModifiedByType": "User"
  },
  "tags": {},
  "type": "Microsoft.KeyVault/vaults"
}
```



Explanation:

- `--enable-rbac-authorization`: Enables **Azure Role-Based Access Control (RBAC)** for secure access control.

Step 5: Configure Workload Identity for Secure Access

```
AzureAD+SahilShivajiPatil@DESKTOP-DBH45CS MINGW64 ~  
$ export SUBSCRIPTION_ID=8809e93f-1006-482f-9f2d-f3b60a303da5  
export RESOURCE_GROUP=keyvault-demo  
export UAMI=azurekeyvaultsecretsprovider-keyvault-demo-cluster  
export KEYVAULT_NAME=aks-demo-sahil2k24  
export CLUSTER_NAME=keyvault-demo-cluster  
  
az account set --subscription $SUBSCRIPTION_ID
```

Set up Environment Variables

```
export SUBSCRIPTION_ID=$(az account show --query id -o tsv)  
export RESOURCE_GROUP="keyvault-demo"  
export UAMI="azurekeyvaultsecretsprovider-keyvault-demo-cluster"  
export KEYVAULT_NAME="aks-demo-abhi"  
export CLUSTER_NAME="keyvault-demo-cluster"
```

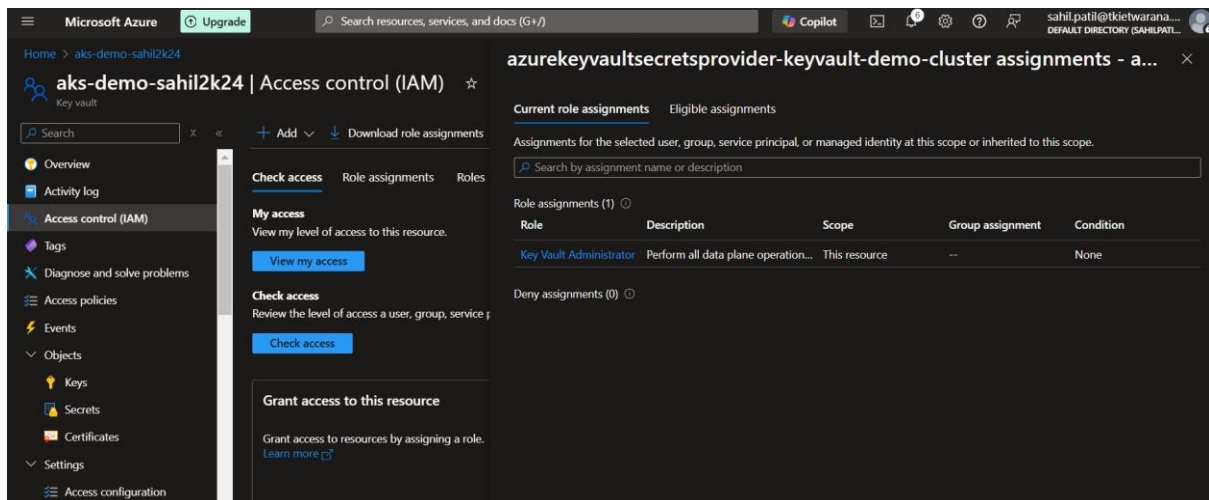
```
az account set --subscription $SUBSCRIPTION_ID
```

Create a Managed Identity

```
AzureAD+SahilShivajiPatil@DESKTOP-DBH45CS MINGW64 ~  
$ az identity create --name $UAMI --resource-group $RESOURCE_GROUP  
  
export USER_ASSIGNED_CLIENT_ID=$(az identity show -g $RESOURCE_GROUP --name $UAMI --query 'clientId' -o tsv)  
export IDENTITY_TENANT=$(az aks show --name $CLUSTER_NAME --resource-group $RESOURCE_GROUP --query identity.tenantId -o tsv)  
{  
  "clientId": "bd86701a-43f5-4736-914e-c431fa53efef",  
  "id": "/subscriptions/8809e93f-1006-482f-9f2d-f3b60a303da5/resourcegroups/keyvault-demo/providers/Microsoft.ManagedIdentity/userAssignedIdentities/azurekeyvaultsecretsprovider-keyvault-demo-cluster",  
  "location": "eastus",  
  "name": "azurekeyvaultsecretsprovider-keyvault-demo-cluster",  
  "principalId": "c389a32e-d5e4-4e3e-b789-85c995ba863e",  
  "resourceGroup": "keyvault-demo",  
  "systemData": null,  
  "tags": {},  
  "tenantId": "ec3036d8-84df-4360-8a91-a7746c320cef",  
  "type": "Microsoft.ManagedIdentity/userAssignedIdentities"  
}
```

```
az identity create --name $UAMI --resource-group $RESOURCE_GROUP  
  
export USER_ASSIGNED_CLIENT_ID=$(az identity show -g $RESOURCE_GROUP --name $UAMI --query 'clientId' -o tsv)  
  
export IDENTITY_TENANT=$(az aks show --name $CLUSTER_NAME --resource-group $RESOURCE_GROUP --query identity.tenantId -o tsv)
```

Assign Key Vault Access Role



```
export KEYVAULT_SCOPE=$(az keyvault show --name $KEYVAULT_NAME --query id -o tsv)
```

```
az role assignment create --role "Key Vault Administrator" --assignee $USER_ASSIGNED_CLIENT_ID --scope $KEYVAULT_SCOPE
```

Step 6: Create a Service Account for Kubernetes Pods

```
export SERVICE_ACCOUNT_NAME="workload-identity-sa"
export SERVICE_ACCOUNT_NAMESPACE="default"
cat <<EOF | kubectl apply -f -
apiVersion: v1
kind: ServiceAccount
metadata:
  annotations:
    azure.workload.identity/client-id: ${USER_ASSIGNED_CLIENT_ID}
  name: ${SERVICE_ACCOUNT_NAME}
  namespace: ${SERVICE_ACCOUNT_NAMESPACE}
EOF
```

Setup Federation

```
export AKS_OIDC_ISSUER="$(az aks show --resource-group $RESOURCE_GROUP --name $CLUSTER_NAME --query "oidcIssuerProfile.issuerUrl" -o tsv)"
```

```
export FEDERATED_IDENTITY_NAME="aksfederatedidentity"
```

```
az identity federated-credential create --name $FEDERATED_IDENTITY_NAME \
--identity-name $UAMI --resource-group $RESOURCE_GROUP --issuer
${AKS_OIDC_ISSUER} \

--subject
system:serviceaccount:${SERVICE_ACCOUNT_NAMESPACE}:${SERVICE_ACCOUNT_
NAME}
```

Step 7: Create a Secret Provider Class

```
azureAD@sahilshivajiPatil@DESKTOP-DBH45CS MINGW64 -
$ kubectl get pods -n kube-system -l 'app in (secrets-store-csi-driver,secrets-store-provider-azure)' -o wide
NAME                                READY   STATUS    RESTARTS   AGE   IP              NODE                                NOMINATED NODE   READINESS GATE
aks-secrets-store-csi-driver-x7rkc   3/3     Running   0           12m   10.244.0.19     aks-nodepool1-15605564-vmss000000 <none>           <none>
aks-secrets-store-provider-azure-v1jbf 1/1     Running   0           12m   10.224.0.4      aks-nodepool1-15605564-vmss000000 <none>           <none>
```

```
cat <<EOF | kubectl apply -f -
apiVersion: secrets-store.csi.x-k8s.io/v1
kind: SecretProviderClass
metadata:
  name: azure-kvname-wi
spec:
  provider: azure
  parameters:
    usePodIdentity: "false"
    clientID: "${USER_ASSIGNED_CLIENT_ID}"
    keyvaultName: ${KEYVAULT_NAME}
  objects: |
    array:
      - |
        objectName: secret1
        objectType: secret
        objectVersion: ""
    tenantId: "${IDENTITY_TENANT}"
EOF
```


Step 8: Deploy a Sample Pod to Access Secrets

```
cat <<EOF | kubectl apply -f -  
apiVersion: v1  
kind: Pod  
metadata:  
  name: busybox-secrets-store-inline-wi  
spec:  
  serviceAccountName: "workload-identity-sa"  
  containers:  
    - name: busybox  
      image: registry.k8s.io/e2e-test-images/busybox:1.29-4  
      command: ["/bin/sleep", "10000"]  
      volumeMounts:  
        - name: secrets-store  
          mountPath: "/mnt/secrets-store"  
          readOnly: true  
  volumes:  
    - name: secrets-store  
      csi:  
        driver: secrets-store.csi.k8s.io  
        volumeAttributes:  
          secretProviderClass: "azure-kvname-wi"  
EOF  
Verify:  
kubectl exec busybox-secrets-store-inline-wi -- ls /mnt/secrets-store/
```

Step 9: Cleanup

```
az group delete --name keyvault-demo
```

This concludes the integration of **Azure Key Vault with AKS using the Secrets Store CSI Driver** for secure secrets management.